Scribble to Digital

Done By:

Team ID : SWTID-2026-3543

TEAM MEMBERS:
PRIYADHARSHINI B
PRAVEEN K
FARTHIMA FARHANA V A
BALAJI G
MAHESHWARI J

Project Description

Turning handwritten notes into digital text is a frequent challenge for both students and working professionals. Issues such as poor lighting, difficult-to-read handwriting, and a mix of content like notes and task lists often reduce efficiency and make manual rewriting necessary.

Scribble to Digital is an AI-based solution designed to transform unorganized handwritten notes into clear and well-structured digital text. The system improves the quality of uploaded images, extracts the text using OCR, and utilizes Generative AI to refine grammar and automatically identify and extract To-Do tasks.

The application is built using Streamlit for the frontend interface, while Python handles the backend processing.

Use Case Scenarios

Scenario 1: Handwritten Note Digitization with Image Enhancement

Users upload a photo of messy handwritten notes through the Streamlit interface. The system internally enhances the image using OpenCV by converting it to grayscale, increasing brightness and contrast, reducing noise, and applying thresholding techniques.

After enhancement, the improved image is passed to the OCR engine for accurate text extraction. This process ensures better readability and higher text recognition accuracy even for low-quality or unclear handwriting.

The output includes:

Raw OCR extracted text
Improved clarity for further AI processing

Scenario 2: Intelligent Text Correction and Contextual Reconstruction

After OCR extraction, the raw text may contain spelling mistakes, missing letters, and broken words due to handwriting variations.

The Generative AI model analyzes the extracted text and:
Corrects spelling errors
Improves grammar and sentence structure
Reconstructs incomplete or unclear words using contextual understanding
Formats the text into readable paragraphs

This ensures that messy handwritten content is converted into professional and well-structured digital text.

Scenario 3: Automatic To-Do Extraction and Task Identification

Handwritten notes often contain a mixture of general information and actionable tasks. Identifying tasks manually can be time-consuming.

The AI system analyses sentence intent and:
Detects task-oriented statements
Extracts actionable items
Organizes them into a structured bullet-point To-Do list

This allows users to quickly identify important tasks separately from general notes, improving productivity and task management.

Architecture Overview

Scribble to Digital is an intelligent AI-powered web application designed to convert handwritten notes into clean, structured digital text. The platform is built using the Streamlit framework, enabling rapid development of an interactive user interface.

The system integrates:
Streamlit for frontend and application logic
OpenCV for image preprocessing and enhancement
OCR Engine (Tesseract/EasyOCR) for text extraction

Google Gemini 2.5 Flash (Generative AI) for contextual correction, formatting, and task extraction

The platform processes handwritten images, enhances clarity, extracts text, corrects errors using AI, and optionally generates structured outputs such as formatted notes and to-do lists.

Core Technologies

Streamlit: Interactive web application framework for UI, file uploads, and result display
Google Gemini 2.5 Flash: AI model for contextual understanding, grammar correction, and smart task extraction
OpenCV: Image preprocessing (grayscale conversion, noise removal, thresholding)
OCR (Tesseract/EasyOCR): Converts enhanced handwritten images into raw digital text
Python: Core programming language
ReportLab / Python-DOCX: Optional export to PDF and Word formats

Component-Wise Architecture

Pre-requisites

Google AI Studio Account Setup – To generate Gemini API Key
Python 3.8+ Installed
Streamlit Framework Installed

Required Python Libraries
- opencv-python
- pytesseract / easyocr
- google-generativeai
- pillow
- reportlab (for PDF export)
- python-docx (for Word export)

Tesseract OCR Installed (if using pytesseract)

Project Workflow

1. Gemini AI Setup & Initialization

Activity 1.1: Set up Google AI Studio with Gemini API

Create or sign in to Google AI Studio account
Generate API key for Gemini model access
Enable Gemini 2.5 Flash model

Test API integration using a sample prompt

Activity 1.2: Configure API Keys and Environment

Store GEMINI_API_KEY securely in .env file
Load environment variables using python-dotenv
Configure model parameters:
- Temperature
- Max output tokens
- Response format

Validate API connectivity inside Streamlit app

Activity 1.3: Design AI Prompts for Note Processing

Create prompt template for:
- Grammar correction
- Sentence reconstruction
- Formatting messy text into structured paragraphs

Design separate prompt for:
- To-do task extraction
- Task prioritization (if needed)

Test prompt effectiveness with sample handwritten text

2. Core Functionality Development

Activity 2.1: Design Streamlit App Structure

Create modular project structure:

Configure Streamlit layout:
Title & description
File uploader
Processing buttons

Output display sections

Activity 2.2: Implement Image Processing Module

Accept image upload (JPG/PNG)
Apply preprocessing using OpenCV:
- Convert to grayscale
- Adjust brightness & contrast
- Remove noise
- Apply thresholding

Display enhanced image preview in Streamlit

Activity 2.3: Implement OCR Extraction

Integrate Tesseract or EasyOCR
Extract raw handwritten text
Clean unnecessary symbols
Display raw OCR output for comparison

Activity 2.4: AI-Based Contextual Processing

Send OCR text to Gemini API
Perform:
- Grammar correction
- Sentence completion
- Context-based word guessing
- Formatting into structured notes

Extract action items separately
Return:
- Clean digital text
- Structured to-do list

3. Streamlit UI Implementation

Activity 3.1: Build Interactive UI Components

st.file_uploader() for image upload
st.image() for displaying uploaded & enhanced image
st.button() for processing steps
st.spinner() for loading indication
st.text_area() for displaying extracted text

Activity 3.2: Display Structured Results

Section 1: Enhanced Image
Section 2: Raw OCR Text
Section 3: AI-Corrected Digital Notes
Section 4: Extracted To-Do Tasks (bullet format)

Optional:
Checkbox to show/hide raw OCR
Download buttons for final output

Activity 3.3: Add Export Functionality

Generate downloadable:
- PDF (using ReportLab)
- Word file (using python-docx)

Use st.download_button()
Allow preview before download

4. Testing & Optimization

Activity 4.1: Functional Testing

Test with:
- Clear handwriting
- Messy handwriting
- Mixed text + tasks
- Bullet points

Validate OCR accuracy
Validate AI corrections

Activity 4.2: Evaluate AI Response Quality

Check:

- Grammar correction accuracy
- Logical sentence flow
- Proper formatting
- Correct task extraction

Ensure no hallucinated content

Activity 4.3: Performance Optimization

Optimize image preprocessing steps
Reduce API token usage
Add error handling for:

- Empty image
- API failure
- OCR failure

Improve prompt clarity

Activity 4.4: Ensure Export Quality

Validate PDF formatting
Check Word document structure
Ensure file compatibility
Test downloads across browsers

MILESTONE 1: Generative AI Initialization

Project: Scribble to Digital (Streamlit Framework)

In this foundational stage, the objective is to establish and configure the AI infrastructure required to power the **Scribble to Digital** application.

The platform uses **Gemini 2.5 Flash**, Google's fast and efficient multimodal model, to:
Correct messy handwritten OCR text
Improve grammar and sentence flow
Reconstruct incomplete words
Extract structured to-do tasks
Convert handwritten notes into clean digital format

Activity 1.1: Set up Google AI Studio and Enable Gemini API

1. **Sign in to Google AI Studio**

Visit: https://makersuite.google.com/app/apikey
Sign in with your Google account
Accept terms and conditions

2. **Create API Key**

Click **"Create API Key"**
Copy the generated API key
Store it securely in a .env file

3. **Select Model**

Choose: **Gemini 2.5 Flash**

Model Features:
Fast response time
Efficient token usage
Suitable for text correction & formatting
Free tier supports multiple requests per minute

Best for:
OCR text cleaning
Contextual rewriting
Structured note generation

Activity 1.2: Configure Environment and Test Connection (Streamlit Setup)

Step 1: Create. env File

Inside your project folder:

Add:

Step 2: Install Required Dependencies

Libraries Used:

streamlit → UI framework
google-generativeai → Gemini API integration
python-dotenv → Load environment variables
opencv-python → Image preprocessing
pytesseract → OCR text extraction
pillow → Image handling

Step 3: Test Gemini API Connection in Streamlit

Create a simple test inside app.py:

Activity 1.3: Validate Connectivity and Response Quality

Test Case 1: Grammar Correction
Input:

Expected Output:

Test Case 2: Structured Note Formatting
Input:

Expected Output:
Formatted paragraph

Bullet list tasks extracted

MILESTONE 2: Core Functionalities Development (Streamlit)

This milestone focuses on building the complete working application using **Streamlit + Gemini 2.5 Flash**.

The goal is to:
Upload handwritten note images
Extract text using OCR
Clean & correct text using Gemini
Extract tasks automatically
Display structured digital output
Enable download options

File Explorer Structure

Activity 2.1: Streamlit Initialization and Import Libraries
Import Required Libraries
In app.py:

Configure Gemini Model

Activity 2.2: OCR Processing Module

Create utils/ocr_processor.py

Purpose:

Convert image to grayscale
Improve brightness
Remove noise
Extract raw OCR text

Activity 2.3: Gemini AI Processing Module

Create utils/ai_processor.py

AI Responsibilities:

- Grammar correction
- Sentence restructuring
- Context guessing
- To-Do extraction

Activity 2.4: Streamlit Main UI Logic

Inside app.py

Activity 2.5: Export Functionality

Create utils/file_export.py
Save as TXT

In app.py:

MILESTONE 3: Backend – Flask Integration (app.py)

This milestone focuses on implementing the complete Flask backend for the **Scribble to Digital** application, including OCR processing, GenAI integration, task extraction, error handling, and export functionality.

Activity 3.1: Complete Route Definitions

All routes are properly defined with:

1. Input Validation

- Validates uploaded image file type (.jpg, .png, .jpeg)
- Checks for empty file upload
- Validates file size limit
- Ensures image can be processed

2. Error Handling

- Handles OCR processing errors
- Handles Gemini API failures
- Returns proper JSON error responses
- Prevents server crash on invalid input

3. Logging

- Logs image upload events

Logs OCR extracted text
Logs GenAI responses
Logs export operations
Debug logs for development mode

4. Response Formatting

Structured JSON response:

Activity 3.2: Modular Architecture

Main Application Structure

Separation of Concerns

ocr_service.py → Handles image preprocessing & Tesseract
ai_service.py → Handles contextual correction & to-do extraction
export_service.py → Handles file generation
app.py → Controls routes & server startup

Flask Server Initialization (app.py)

Activity 3.3: Export Implementation

Complete implementation of:

1. PDF Generation

Clean formatted digital notes
Structured to-do list section
Professional layout
Downloadable file

2. Word Document Export (.docx)

Heading styles
Bullet points for tasks

Editable format

3. CSV Export (Task List)

Task column
Status column (Pending/Done)
Compatible with Excel

4. File Cleanup & Management

Temporary file storage
Auto-delete after download
Secure file handling

MILESTONE 4: UI Development

This milestone focuses on building a modern, responsive, and interactive frontend interface using Streamlit for the Scribble to Digital application.

The UI allows users to:

Upload handwritten notes
Process them using OCR + Generative AI
View structured digital output
Download formatted results

Activity 4.1: Landing Page Development

Home Page: Transform Your Notes into Digital Intelligence

·· Hero Section

Title:
✍️ Transform Your Handwritten Notes into Smart Digital Content

Subtitle:
Convert messy notes into clean text and structured to-do lists using OCR and AI.

·· Feature Cards Section

Using Streamlit columns layout:

1. Upload Notes

Accept handwritten image
Supports JPG, PNG, JPEG
Instant preview

2. Smart Correction

Fixes spelling mistakes
Uses contextual AI logic
Improves readability

3. To-Do Extraction

Identifies action items
Converts tasks into list format
Separates notes from tasks

Activity 4.2: Processing Page (Main Generator Page)

Handwritten Notes Processing Page

Input Fields:

Image Upload Section
Convert Button
Reset Button

UI Layout (Streamlit Flow)

Activity 4.3: UI Components Implementation (Streamlit Code Structure)

Sidebar Navigation (Multi-page Design)

Home Page Layout

Processing Page Layout

Activity 4.4: Export Interface

Export Options:
Download Clean Text (.txt)
Download Task List (.csv)
Future enhancement: PDF export

Activity 4.5: UI Design Principles Applied

Clean card-style sections
Proper spacing
Clear section separation
User feedback (spinner, success message)
Mobile responsive (Streamlit auto-adjusts)
Logical workflow design

Complete User Interaction Flow

User opens application.
Views landing page with features.
Navigates to processing page.
Uploads handwritten image.
Clicks convert.
Views:
- Clean text
- Extracted tasks

Downloads results.

MILESTONE 5: Testing & Optimization

Scribble to Digital – Handwritten Notes Digitizer

This milestone focuses on comprehensive testing, quality validation, and performance optimization of the Scribble to Digital application to ensure accuracy, reliability, and smooth user experience.

Activity 5.1: Comprehensive Testing

Test Scenarios

Test Scenario 1: Clear Handwriting

High-resolution image
Proper lighting
Neatly written text

Expected Result:
Accurate OCR text extraction
Proper grammar correction using AI
Correct identification of tasks

Outcome:
The system successfully extracted clean text and correctly separated action items into a to-do list.

Test Scenario 2: Medium Quality Handwriting

Slight shadows
Minor spelling distortions
Mixed notes and tasks

Expected Result:

OCR extracts readable text
AI corrects spelling using contextual logic
Tasks are properly separated

Outcome:
The AI improved sentence clarity and successfully identified most tasks.

Test Scenario 3: Poor Quality Image

Blurry text
Low brightness
Irregular handwriting

Expected Result:

Partial text extraction
Context-based correction
Limited but usable output

Outcome:
Image enhancement improved visibility, and the AI generated partially corrected readable text.

Test Scenario 4: Notes Without Tasks

Informational notes only
No action keywords

Expected Result:

Clean digital text
No tasks detected

Outcome:
System correctly displayed cleaned notes and returned “No tasks detected”.

Test Scenario 5: Bullet Point Notes

Notes containing dashes or numbers

Clearly defined tasks

Expected Result:
Structured to-do list
Proper formatting

Outcome:
Tasks were correctly identified and formatted as a digital list.

Activity 5.2: Quality Evaluation

The following aspects were evaluated:

OCR Accuracy
The percentage of correctly recognized words from handwritten input.

Contextual Correction
Improvement in grammar, clarity, and sentence meaning after AI processing.

To-Do Extraction Accuracy
Correct identification of action items without duplication or irrelevant content.

Format Consistency
Proper separation of clean text and to-do list sections.

Export Quality
Verification that downloaded files (TXT, CSV, PDF if enabled) are correctly formatted and usable.

Activity 5.3: Performance Optimization

Several improvements were implemented to enhance system performance:
Optimized image preprocessing to reduce processing time.
Improved prompt structure to reduce unnecessary API response length.
Added input validation for file type and size.
Implemented loading spinner to improve user experience.
Prevented automatic reprocessing on page refresh in Streamlit.
Handled API and OCR errors gracefully with user-friendly messages.

Stress Testing

Multiple images were uploaded sequentially to verify stability.
Large image files were tested to ensure the system does not crash.
System remained stable under continuous usage.

Edge Case Handling

Blank image upload

Non-text image upload
OCR failure
Gemini API failure
Internet connectivity issues
All cases were handled with appropriate error messages.

Conclusion

Scribble to Digital transforms traditional handwritten note-taking into an intelligent, AI-powered digital workflow. By combining OCR technology with advanced generative AI, the system accurately extracts handwritten content, enhances readability through contextual correction, and intelligently identifies actionable tasks. The integration of image preprocessing, structured AI prompts, and a responsive Streamlit frontend ensures a seamless and user-friendly experience.

From image enhancement and text recognition to contextual understanding and to-do extraction, every component of Scribble to Digital is designed with a focus on accuracy, efficiency, and reliability. The system not only digitizes messy notes but also adds value by organizing content into clean text and structured task lists, saving users time and effort.

With modular architecture, effective error handling, and optimized performance, Scribble to Digital demonstrates how AI can bridge the gap between handwritten creativity and digital productivity. Whether used by students, professionals, or teams, the application serves as a smart assistant that enhances organization, improves clarity, and streamlines everyday workflows.

Component	**Description**

Streamlit Frontend UI	Provides a simple web interface for users to upload handwritten images. Displays enhanced images, extracted text, corrected content, and to-do lists dynamically.
Image Processing Layer (OpenCV)	Enhances uploaded images by converting to grayscale, adjusting brightness/contrast, reducing noise, and applying thresholding to improve OCR accuracy.
OCR Layer	Extracts raw text from the processed image. Handles handwriting recognition and converts image content into machine-readable text.
Generative AI Layer (Gemini 2.5 Flash)	Processes extracted text to: • Correct spelling and grammar • Reconstruct incomplete words • Format into structured paragraphs • Extract actionable to-do items
Export Module	Converts AI-processed content into downloadable formats such as PDF or Word documents.